Tugas Besar Pengolahan Citra
Smart Farming Dalam Deteksi Kualitas Tomat

1122008 - Steven Filbert N
1122049 - Erwin Dosantos
1123021 - Josef Charend Alvaro
1123049 - Adrian Renard Kristanto

Link Collab :
https://colab.research.google.com/drive/170fYZlWjOgLtwTedrVvfpL9E9KuajaBk?usp=sharing

Identifikasi Masalah

Ketika musim panen tiba (dalam hal ini panen tomat), seorang petani terlebih dahulu perlu menyortir dan memisahkan hasil panen tomat yang bagus (matang, mulus, merah) dengan tomat yang kurang bagus (busuk, ada bintik hitam, masih hijau). Memisahkan dengan tangan manusia itu lama, melelahkan, kadang tidak konsisten bahkan bisa berbeda subjektivitas.

Oleh karenanya, untuk mengatasi permasalahan itu biarkan komputer yang bekerja. Dengan bantuan machine learning, komputer akan:

1. mengambil gambar buah,
2. mengolah gambar,
3. mengambil ciri warna / tekstur, dan
4. mengklasifikasikan kualitas buah.

Bantuan ini diharapkan mampu mempercepat proses cek kualitas serta penilaian konsisten yang lebih objektif.

Tahapan Penyelesaian

Tahap 1: Ambil Dataset

Dataset yang digunakan diambil dari website Kaggle. Berikut link terkait.
https://www.kaggle.com/datasets/enalis/tomatoes-dataset
Dataset dibagi jadi 2 fungsi, yaitu untuk training dan test model machine learning nya. Masing-masing fungsi telah diisikan gambar-gambar dalam 4 kategori yang terdiri dari Unripe (tidak matang), Ripe (matang), Old (terlalu matang), Damaged (rusak).

```
train_path = "Dataset/train"
val_path = "Dataset/val"
train_images, train_labels = load_dataset(train_path)
```

```
val_images, val_labels = load_dataset(val_path)
```

```
def load_dataset(base_path):

    images = []
    labels = []

    classes = ["Damaged","Old","Ripe","Unripe"]

    for label in classes:

        folder = os.path.join(base_path,label)

        if not os.path.exists(folder):
            continue

        for file in os.listdir(folder):

            img_path = os.path.join(folder,file)

            img = cv2.imread(img_path)

            if img is None:
                raise ValueError(f"Gambar tidak terbaca: {path}")

             img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            img = cv2.resize(img,(100,100))

            images.append(img)
            labels.append(label)

    return images, labels
```

Setiap fungsi (train & val) akan memanggil fungsi load_dataset. Di load_dataset, library os digunakan untuk memeriksa folder dan image tiap classes ada atau tidak. cv2.cvtColor(img, cv2.COLOR_BGR2RGB) mengubah urutan kanal warna dari gambar yang semula BGR (Blue-Green-Red) menjadi RGB (Red-Green-Blue). Tujuannya untuk menghindari kesalahan visualisasi dan konsistensi dalam ekstraksi fitur, berhubung dengan penggunaan skimage untuk HOG dan praktik terbaik adalah RGB

Image yang ditemukan akan diubah ukurannya menjadi 100x100 px menggunakan library cv2 supaya jumlah fiturnya nanti tidak akan berbeda. Ukuran gambar di-resize menjadi 100x100 karena ukuran tersebut dianggap cukup untuk mempertahankan informasi penting

pada objek tomat seperti bentuk, tekstur, dan warna, namun tetap ringan untuk diproses oleh komputer. Jika ukuran gambar terlalu besar, proses ekstraksi fitur HOG dan training SVM akan menjadi lebih lambat serta menghasilkan jumlah fitur yang sangat banyak. Sebaliknya, jika ukuran terlalu kecil maka detail penting seperti bercak kerusakan, tekstur permukaan, atau perbedaan kematangan tomat dapat hilang. Oleh karena itu ukuran 100x100 dipilih sebagai kompromi antara kualitas informasi gambar dan efisiensi komputasi. Selain itu, penggunaan ukuran yang sama pada seluruh dataset juga membuat jumlah fitur yang dihasilkan konsisten sehingga mempermudah proses training model machine learning. Outputnya adalah seluruh image beserta label masing-masingnya dari salah satu classes.

Tahap 2: Ekstraksi Fitur

```
X_train = extract_features(train_images)
X_val = extract_features(val_images)

y_train = train_labels
y_val = val_labels
```

```
def extract_features(images):

    features = []

    for img in images:

        gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

        hog_feature = hog(
            gray,
            orientations=9,
            pixels_per_cell=(8,8),
            cells_per_block=(2,2),
            block_norm='L2-Hys'
        )

        # ===== RGB FEATURE
        mean_color = np.mean(img, axis=(0,1))
        std_color  = np.std(img, axis=(0,1))

        color_feat = np.concatenate([mean_color, std_color])
        final_feat = np.concatenate([hog_feature, color_feat])
        features.append(final_feat)
```

```
        features.append(hog_feature)

    return np.array(features)
```

Setiap gambar dilakukan akan ekstraksi fitur dari gambar 3 channel RGB menjadi array numerik. Hal ini disebabkan model machine learning Support Vector Machine (SVM) yang digunakan tidak bisa membaca gambar langsung, jadi gambar harus diubah menjadi angka (feature vector).

Mula-mula, setiap gambar berwarna diubah jadi grayscale karena Histogram of Oriented Gradients (HOG) bekerja pada gradien intensitas, bukan warna sehingga 1 channel warna sudah cukup. Selanjutnya, fungsi hog() mengambil pola arah tepi gambar.

Cara kerjanya:

1. gambar dibagi menjadi cell kecil,
2. tiap cell dihitung arah gradien,
3. arah gradien dimasukkan ke histogram,
4. histogram digabung menjadi fitur.

- orientations = 9 artinya histogram memiliki arah gradien dari 0° sampai 180° yang dibagi jadi 9 bagian. Untuk tugas besar ini dengan tomat sebagai objek identifikasi, 9 sudah cukup menangkap bentuk.
- pixels_per_cell = (8,8) artinya gambar dibagi jadi cell-cell yang masing-masing berisi 8x8 px (standar umum di HOG, tidak terlalu besar, tidak terlalu kecil). Dalam banyak penelitian computer vision:
  - 4x4 → terlalu sensitif noise
  - 8x8 → paling stabil
  - 16x16 → detail mulai hilang

  Jadi 8x8 dipilih karena empirically stabil. Berikut adalah perhitungan pembagian gambarn 100x100 px nya jadi 1 cell 8x8:

  - 100 / 8 = 12 cell
  - 12x8 = 96 -> area = 96x96 px

  Ada sisa 100 - 96 = 4 pixel pada sisi kanan dan bawah yang tidak digunakan dalam perhitungan HOG. Jadi HOG hanya membaca 96×96 bagian kiri atas gambar.

  Biasanya ini tidak menjadi masalah signifikan, karena 4 pixel dari 100 = hanya 4% serta edge utama objek masih tetap terbaca. Apalagi ini hanya klasifikasi tomat, jadi yang dibaca HOG adalah tekstur kulit, kontur tomat, pola kerusakan yang semuanya masih berada di area utama gambar.

- cells_per_block = (2,2) (1 block = 2(8x8) px cell= 16x16 px) untuk normalisasi fitur supaya tidak sensitif terhadap pencahayaan maupun bayangan yang bisa menyebabkan banyak detail hilang.
- block_norm='L2-Hys' agar fitur tidak terlalu dipengaruhi pencahayaan serta lebih stabil berkat L2-Hys sebagai metode standar pada HOG. Setelah HOG dihitung, hasilnya adalah vektor 1 dimensi dimasukkan ke list features. Vector ini mewakili pola bentuk dan tekstur gambar. Contoh: hog_feature = [0.12,0.23,0.44,0.19,0.55, ...].
- Output yang dikembalikan berupa numpy array numerik supaya bisa digunakan SVM.
- mean_color digunakan untuk menghitung rata-rata warna RGB pada seluruh gambar. Hasilnya berupa 3 nilai yaitu rata-rata Red, Green, dan Blue. Fitur ini membantu model mengenali dominasi warna tomat, misalnya tomat matang cenderung memiliki warna merah lebih tinggi dibanding tomat mentah.
- std_color digunakan untuk menghitung standar deviasi warna RGB. Tujuannya untuk melihat seberapa besar variasi warna pada gambar. Jika variasi warna tinggi maka kemungkinan terdapat bercak, bayangan, atau kerusakan pada permukaan tomat.
- color_feat = np.concatenate([mean_color, std_color]) np.concatenate() digunakan untuk menggabungkan fitur rata-rata warna dan standar deviasi warna menjadi satu vector fitur warna. Hasil akhirnya berisi 6 nilai numerik:
  - 3 nilai mean RGB
  - 3 nilai std RGB
- final_feat = np.concatenate([hog_feat, color_feat]) np.concatenate() kembali digunakan untuk menggabungkan fitur HOG dan fitur warna menjadi satu feature vector akhir. Penggabungan ini dilakukan karena SVM hanya dapat menerima satu vector numerik untuk setiap gambar. Dengan kombinasi tersebut, model dapat mengenali bentuk, tekstur, dan warna tomat secara bersamaan sehingga klasifikasi menjadi lebih baik.

Kemudian setelah itu digunakan juga RGB Feature yang memiliki fungsi untuk mengambil informasi warna dari gambar agar model SVM tidak hanya mengenali bentuk objek, tetapi juga mengenali karakteristik warnanya. Nantinya akan dihitung juga rata-rata warna RGB dan penyebaran / variasi RGB nya.

Tahap 3: Scalling

```
X_train, mean, std = standardize(X_train)

X_val = standardize_val(X_val, mean, std)

def standardize(X):

    mean = np.mean(X, axis=0)

    std  = np.std(X, axis=0) + 1e-8

    return (X - mean) / std, mean, std
```

```
def standardize_val(X, mean, std):

    return (X - mean) / std
```

Scaling digunakan supaya semua fitur punya skala nilai yang lebih seimbang sebelum masuk ke proses training SVM. Soalnya nilai dari fitur HOG dan fitur RGB berbeda cukup jauh. Fitur HOG biasanya nilainya kecil seperti 0.12 atau 0.45, sedangkan fitur warna RGB bisa sampai 100 atau 200. Kalau langsung dipakai tanpa scaling, model SVM cenderung lebih fokus ke fitur yang nilainya besar dan mengabaikan fitur lain yang sebenarnya juga penting.

Karena itu dilakukan scaling dengan cara mengurangi setiap nilai fitur dengan rata-ratanya lalu dibagi standar deviasinya menggunakan rumus (X - mean) / std. Hasilnya, semua fitur jadi berada pada skala yang lebih mirip. Output dari scaling tetap berupa array numerik (numpy array), hanya saja nilainya sudah dinormalisasi dan biasanya berada di sekitar angka kecil dengan rata-rata mendekati 0. Dengan scaling, proses training jadi lebih stabil, perhitungan SVM lebih optimal, dan model bisa belajar dari semua fitur dengan lebih seimbang.

Tahap 4: Training Model

```
linear_model = train_svm_linear(X_train,y_train)

rbf_model = train_svm_rbf(X_train,y_train)



def train_svm_linear(X_train,y_train):

    model = SVC(kernel='linear', class_weight='balanced')

    model.fit(X_train,y_train)

    return model



def train_svm_rbf(X_train,y_train):

    model = SVC(kernel='rbf', class_weight='balanced')
```

```
    model.fit(X_train,y_train)

    return model
```

Di tahap ini, model SVM (class SVC (Support Vector Classifier)) akan belajar dari dataset train dengan 2 macam kernel (Linear & RBF). Konsepnya adalah mencari garis yang memisahkan data (decision boundary) dengan jarak (margin) terbesar. SVC dengan kernel Linear akan mencoba memisahkan data dengan garis lurus. Berbeda dengan RBF (Radial Basis Function), bisa membuat batas melengkung. Ada penambahan parameter class_weight='balanced' untuk mengatasi imbalance ketika training sehingga sedikit mencegah keberpihakan. Setelahnya decision boundary ditentukan, model.fit(X_train,y_train) dijalankan untuk memulai proses training.

Pemilihan SVM sebagai model didasarkan pada algoritma ini efektif untuk dataset berukuran kecil hingga menengah dan mampu bekerja dengan baik pada data berdimensi tinggi seperti fitur HOG. Selain itu SVM memiliki kemampuan generalisasi yang baik dengan memaksimalkan margin antar kelas sehingga risiko overfitting lebih rendah dibandingkan beberapa model lain seperti decision tree.

Pada tahap training, model SVM menerima dua input utama yaitu X_train dan y_train.

X_train berisi seluruh fitur hasil ekstraksi gambar yang sebelumnya sudah diproses menggunakan HOG dan RGB feature. Jadi gambar tomat tidak langsung dimasukkan ke SVM dalam bentuk image, tetapi sudah diubah menjadi kumpulan angka (feature vector). Bentuk datanya berupa numpy array 2 dimensi, di mana:

- setiap baris mewakili 1 gambar tomat
- setiap kolom berisi nilai fitur dari gambar tersebut

Contohnya seperti:

[

[0.12, -0.44, 1.02, ...],

[0.55, 0.21, 0.88, ...],

[0.91, -0.11, 0.45, ...]

]

Sedangkan y_train berisi label atau kelas dari masing-masing gambar tomat. Bentuknya berupa list atau array string kategori seperti: ["Ripe", "Damaged", "Unripe", "Old"]

Label ini digunakan sebagai jawaban benar saat training supaya model bisa belajar hubungan antara fitur gambar dan kelasnya. Jadi secara sederhana:

- X_train = data fitur numerik hasil olahan gambar
- y_train = label target dari gambar tersebut

Kemudian SVM akan mempelajari pola dari data tersebut untuk mencari pemisah antar kelas tomat.

Tahap 5: Evaluasi/Tes Model

```
evaluate_model(linear_model, X_val, y_val, "Linear")
evaluate_model(rbf_model, X_val, y_val, "RBF")
```

```
def evaluate_model(model, X_test, y_test, model_name):

    predictions = model.predict(X_test)

    acc = accuracy_score(y_test, predictions)

    print("Accuracy :", acc)
    print()

    print("Classification Report")
    print(classification_report(y_test, predictions))

    cm = confusion_matrix(y_test, predictions)

    # membuat visualisasi confusion matrix
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)

    disp.plot(cmap='Blues')
    plt.title("Confusion Matrix - " + model_name)

    # simpan sebagai gambar
    filename = "confusion_matrix_" + model_name + ".png"
    plt.savefig(filename)

    print("Confusion matrix saved as:", filename)

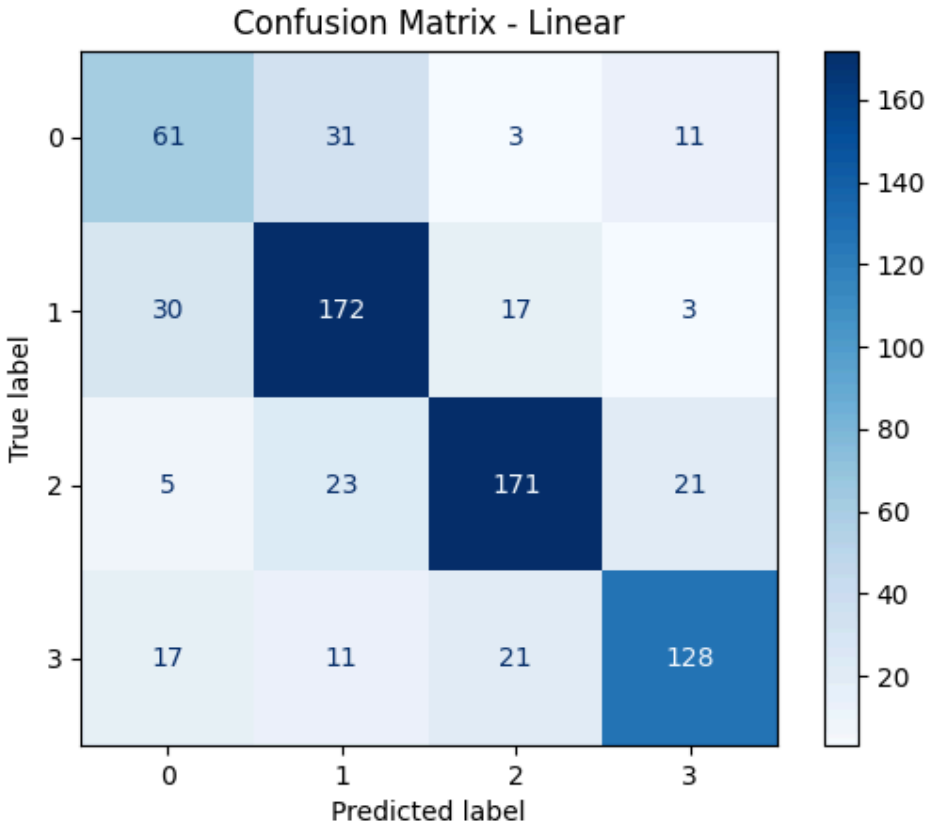
    plt.show()
```

Setelah training, model digunakan untuk prediksi.

- Ketika predictions = model.predict(X_test), model akan mencoba menebak kelas gambar baru.
- Accuracy juga dihitung dengan cara jumlah prediksi benar / total data.
- Classification Report yang meliputi Precision, Recall, dan F1-Score juga ikut ditampilkan.
- Untuk menunjukan detail kesalahan model, confusion matrix dibuat dalam bentuk gambar untuk masing-masing kernel.

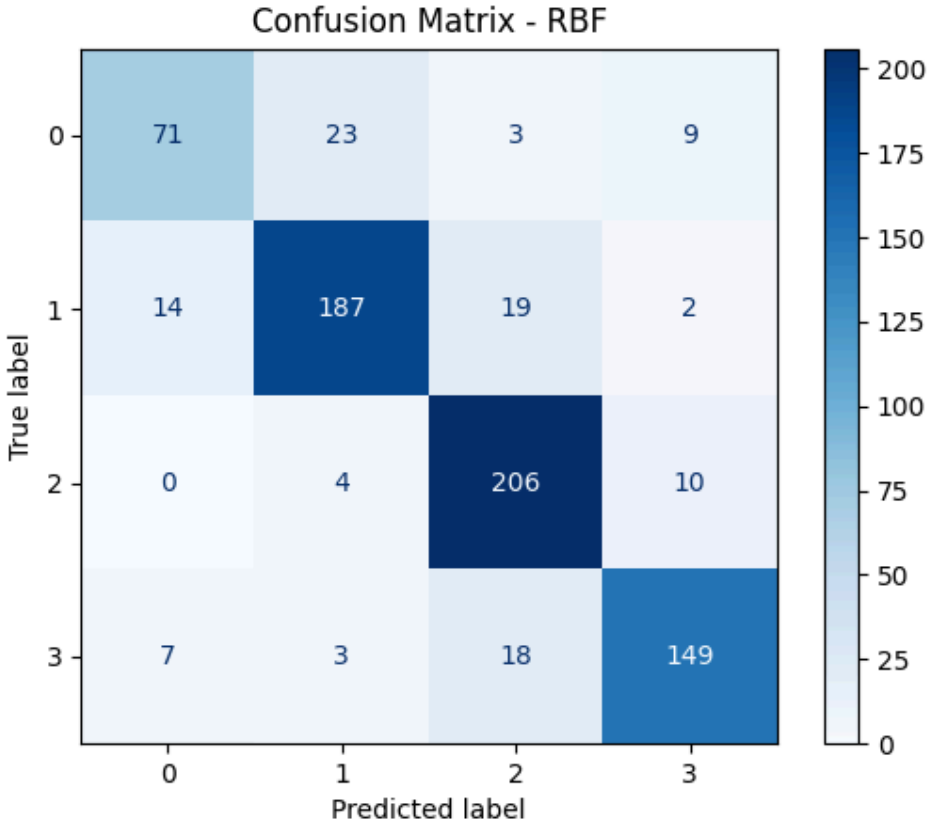
Hasil Confusion Matrix:

- **Linear**



Berdasarkan Confusion Matrix di atas maka bisa disimpulkan accuracy data train dari SVM Linear adalah 0.7337.

- **RBF**



Berdasarkan Confusion Matrix di atas maka bisa disimpulkan accuracy data train dari SVM RBF adalah 0.8455.